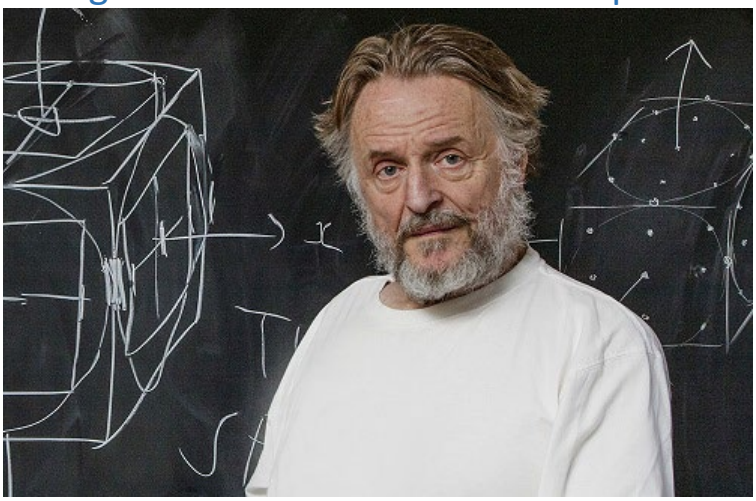
El Juego de la Vida: la batalla de los patrones



Más información sobre este juego: https://youtu.be/OWXD_wJxCKQ

Introducción

El **Juego de la Vida** (en inglés, *Conway's Game of Life*) es un autómata celular ideado en 1970 por el matemático británico **John Horton Conway**, profesor en Cambridge y más tarde en Princeton. Conway buscaba un conjunto de reglas sencillas capaz de producir comportamientos complejos e impredecibles a partir de configuraciones muy simples, inspirándose en trabajos previos sobre autómatas autorreplicantes de John von Neumann.

El juego se dio a conocer al gran público gracias al divulgador **Martin Gardner** en su columna "Mathematical Games" de la revista *Scientific American* en octubre de 1970. El artículo tuvo tanto éxito que generó una auténtica fiebre por buscar y catalogar patrones, mucho antes de que existieran ordenadores personales: los primeros aficionados simulaban las generaciones a mano, sobre tableros de Go.

Lo curioso del Juego de la Vida es que, en su versión original, es un **juego de cero jugadores**: no hay turnos ni decisiones que tomar. El "jugador" solo elige la configuración inicial de células vivas, y a partir de ahí el tablero evoluciona solo, generación tras generación, siguiendo siempre las mismas reglas. A pesar de (o precisamente por) esta simplicidad, se ha demostrado que el Juego de la Vida es **Turing-completo**: con la configuración inicial adecuada se puede construir dentro de él cualquier programa computable, incluyendo puertas lógicas, memorias e incluso réplicas del propio juego.

A lo largo de más de cincuenta años se han catalogado miles de patrones con nombre propio. Algunos ejemplos célebres son las **naves espaciales** ("spaceships") como el *planeador* ("glider"), que se desplaza en diagonal reproduciéndose a sí mismo cada 4 generaciones; los **osciladores** como el "parpadeador" ("blinker"), que alternan entre dos formas; las formas *estables* que nunca cambian, como el "bloque"; y los generadores como el *cañón de planeadores de Gosper*, que emite un planeador nuevo cada 30 generaciones sin destruirse nunca.

En esta práctica vamos a transformar este "juego de cero jugadores" en un juego de dos, en el que cada jugador elige una configuración inicial con un presupuesto limitado de células y compite para conseguir que su patrón sobreviva el mayor número de generaciones posible antes de estabilizarse.

Las reglas del Juego de la Vida

El tablero es una cuadrícula finita de celdas. Cada celda tiene dos estados posibles: viva o muerta. En cada generación, el estado de cada celda para la siguiente generación se calcula a partir de su estado actual y del número de vecinas vivas que tiene entre sus 8 celdas adyacentes (horizontal, vertical y diagonal), siguiendo estas cuatro reglas clásicas (conocidas como B3/S23):

- Una célula **viva** con menos de 2 vecinas vivas muere (por *infrapoblación*).
- Una célula **viva** con 2 o 3 vecinas vivas sigue **viva** en la siguiente generación.
- Una célula **viva** con más de 3 vecinas vivas muere (por *sobrepoblación*).
- Una célula **muerta** con exactamente 3 vecinas vivas **nace** en la siguiente generación.

En nuestra versión, el tablero es finito y de bordes muertos: las celdas fuera de la cuadrícula se consideran siempre muertas.

Nuestra variante: la competición

La partida tiene dos jugadores, cada uno con su propio tablero (independiente del otro, del mismo tamaño), y se juega en dos fases:

- **Fase de colocación:** los jugadores se alternan turnos y, en cada turno, colocan una única célula viva en una casilla libre de su propio tablero. Cada jugador dispone de un **presupuesto máximo de células** (por ejemplo, 10). La fase termina cuando ambos jugadores han agotado su presupuesto.
- **Fase de simulación:** cada tablero evoluciona de forma independiente, generación a generación, siguiendo las reglas del Juego de la Vida, hasta que se **estabiliza**: es decir, hasta que alcanza un estado ya visto anteriormente en esa misma partida (lo que cubre tanto un estado fijo, con 0 células o con formas estables como el bloque, como un ciclo de un oscilador o el momento en que un planeador desaparece por el borde).

Gana la partida el jugador cuyo tablero tarda **más generaciones** en estabilizarse. En caso de empate a generaciones, gana quien tenga **más células vivas** en ese estado final. Para evitar partidas que no terminen nunca (algunos patrones tardan muchísimo en estabilizarse, y unos pocos, en tableros grandes, no lo hacen jamás), se establece también un número máximo de generaciones simuladas; si se alcanza, el tablero se considera estabilizado en ese punto a efectos de la comparación.

Ejemplos de patrones

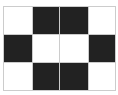
Antes de poneros a diseñar vuestras propias configuraciones iniciales, conviene conocer algunos de los patrones con nombre propio más citados, agrupados según su comportamiento:

Vidas estáticas

Bloque

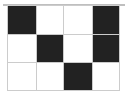


Colmena de abejas



Pan





Bote



Bañera



Osciladores

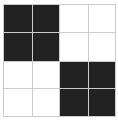
Intermitente (período 2)



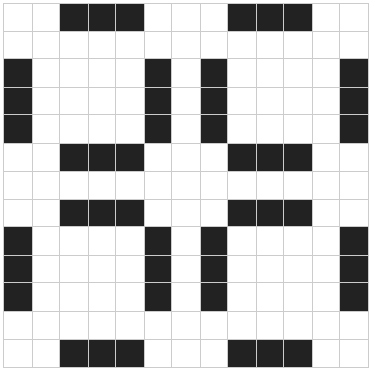
Sapo (período 2)



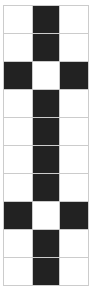
Faro (período 2)



Púlsar (período 3)



Pentadecatlón (período 15)

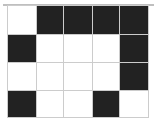


Naves espaciales

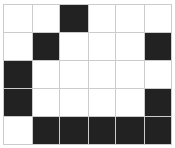
Planeador



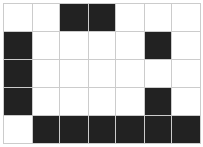
Nave espacial ligera



Nave espacial de peso medio



Nave espacial pesada



Existen numerosos tipos de patrones de células que pueden tener lugar en el juego de la vida.

Implementación del juego

Vamos a construir el juego pieza a pieza, respetando los principios de la programación funcional: datos inmutables, funciones puras y recursión en lugar de bucles con variables mutables. Los objetivos de diseño son los siguientes:

- Separar la lógica del juego (la clase *Game*) de la entrada/salida (las estrategias y el programa principal).
- Construir una estructura de datos genérica y reutilizable (*Matrix[T]*) y especializarla mediante **herencia** para el Juego de la Vida (*GameOfLifeMatrix*).
- Practicar la recursión (incluida la recursión de cola) y el pattern matching sobre *List* dentro de las matrices, y compararla después con las funciones de orden superior que ya trae la librería estándar.
- Modelar los errores con *Either* y *Option* en lugar de con excepciones o con casos que no se controlan.
- Usar traits, case objects y polimorfismo para modelar comportamientos intercambiables (jugador humano o IA).

En nuestra versión, el tablero es siempre de **15x15** y la simulación se detiene como máximo a las **1000 generaciones**.

Organización del código y material de partida

Todo el código va en el paquete *gameoflife* (primera línea de cada fichero: *package gameoflife*), repartido en ficheros de una sola responsabilidad. Os proporcionamos una **plantilla** con estos ficheros, las firmas de todas las funciones con su cuerpo sustituido por *???*, la documentación de cada una y las partes que ya vienen resueltas. Respetad exactamente los nombres y las firmas: las pruebas que os damos dependen de ellas.

Fichero	Contenido
Cell.scala	El tipo *Cell* y sus dos valores, *Alive* y *Dead*.
Position.scala	La case class *Position*.
Matrix.scala	La matriz genérica *Matrix[T]* (recursiva).
GameOfLifeMatrix.scala	El tablero del juego, que hereda de *Matrix[Cell]*.

Fichero	Contenido
Player.scala	El enumerado *Player*, *otherPlayer* y *playerName*.
State.scala	La case class *State* (datos de la fase de colocación).
GameResult.scala	La case class *GameResult* (resultado de una partida).
Strategy.scala	El trait *Strategy* y sus implementaciones *ManualStrategy* y *AIStrategy*.
Game.scala	La clase *Game*, con toda la lógica de la partida.
Main.scala	El programa principal.
Animation.scala	**Ya implementado**: limpiar la consola, dibujar dos tableros juntos y animar la simulación.
MatrixList.scala, *GameOfLifeMatrixList.scala*	Parte opcional: las mismas matrices con los métodos de *List*.

Qué se puede usar en cada parte. En *Matrix* y *GameOfLifeMatrix* el objetivo es practicar la recursión: se escriben con recursión y pattern matching sobre *Nil* y *head :: tail*, sin los métodos de orden superior de *List* (*map*, *filter*, *count*, *partition*, *foldLeft*, *flatMap*, *zip*...) ni el acceso por índice *data(i)*. Sí podéis construir listas con *::* y usar *reverse*, *++*, *length* y *contains*. En el resto de la práctica (*Game*, las estrategias y el programa principal) podéis y debéis usar las funciones de orden superior que necesitéis. Las más útiles son *map*, *foldLeft*, *maxBy*, *count*, *filter*, *zip* y *toMap* de *List*, y *updated* de *Map*. Tenéis una explicación con ejemplos de todas ellas en el libro de Scala 3.

Las células y las posiciones

En vez de representar cada celda con un simple *Boolean*, definimos un tipo propio y cerrado con las dos posibilidades: un *sealed trait* con dos *case object*. Las coordenadas de una celda son una case class con la fila y la columna, empezando ambas en 0. Estos tipos vienen **ya definidos** en la plantilla:

Tipo	Qué es
Cell	Un *sealed trait*: el tipo de una celda. Solo tiene dos valores posibles.
Alive, *Dead*	Dos *case object* que extienden de *Cell*: célula viva y célula muerta.
Position(row: Int, column: Int)	Una case class con la fila y la columna de una celda, empezando ambas en 0.

Al ser *sealed*, el compilador conoce todos los casos de *Cell* y os avisará si un pattern matching se deja alguno. Además, el código se lee solo: una *Matrix[Cell]* no necesita explicación, mientras que una *Matrix[Boolean]* obliga a recordar qué significa *true*.

Nota: Por ser case class, dos *Position* con las mismas coordenadas son iguales (*Position(1, 2) == Position(1, 2)*).

Una clase Matrix genérica (2 ptos)

El tablero se representa como una **matriz bidimensional genérica** que no sabe nada del Juego de la Vida: guarda valores de cualquier tipo *T*. Internamente, los datos se guardan en una única *List* (no en una *List* de *List*) en **orden de filas**: primero toda la fila 0, después toda la fila 1, etc. Así, la posición *(r, c)* ocupa el índice *r * columns + c* de la lista.

Matrix es una clase normal, **no una case class**: sus parámetros se declaran con *val* para que sean accesibles desde fuera y desde las subclases. La plantilla ya incluye el constructor, que comprueba que *data* tiene exactamente *rows * columns* elementos (si no, lanza *IllegalArgumentException*), los métodos privados *index* y *positionOf* y, como ejemplo, uno de los constructores del objeto compañero:

Elemento	Descripción
Matrix[T](rows: Int, columns: Int, data: List[T])	La clase. Sus tres parámetros son *val*. El constructor comprueba que *data* tiene *rows * columns* elementos. **Ya implementado.**
Matrix(rows, columns, data)	Crea una matriz a partir de sus datos, en orden de filas. **Ya implementado** en el objeto compañero, como ejemplo.
index(p: Position): Int	Método privado: el índice de *p* dentro de *data*. **Ya implementado.**
positionOf(i: Int): Position	Método privado, el inverso de *index*: la posición que ocupa el elemento *i* de *data*. **Ya implementado.**

El Companion object. Sus métodos *apply* son «fábricas»: permiten crear una *Matrix* sin *new* (por ejemplo, *Matrix(2, 3, List(1, 2, 3, 4, 5, 6))*) y ofrecer varias formas de construirla, cada una con sus propios parámetros, mientras que la clase solo tiene un constructor. El que crea la matriz a partir de sus datos viene hecho como ejemplo; los otros dos forman parte de la práctica.

En los ejemplos de la tabla, *m* es *Matrix(2, 3, List(1, 2, 3, 4, 5, 6))*, es decir, la matriz con las filas *1 2 3* y *4 5 6*:

Método	Qué debe cumplir	Ejemplo
Matrix(rows: Int, columns: Int, defaultValue: T): Matrix[T] (Companion object)	Una matriz de *rows* x *columns* con *defaultValue* en todas las celdas.	*Matrix(2, 3, 0).data* *→ List(0, 0, 0, 0, 0, 0)*
Matrix(rows: Int, columns: Int, defaultValue: T, marked: List[Position], markedValue: T): Matrix[T] (Companion object)	Una matriz con *defaultValue* en todas las celdas salvo en las posiciones de la lista *marked*, que tienen *markedValue*.	*Matrix(2, 3, 0, List(Position(0, 2), Position(1, 0)), 7).data* *→ List(0, 0, 7, 7, 0, 0)*
inside(p: Position): Boolean	Verdadero si la posición está dentro de los rangos de la matriz.	*m.inside(Position(1, 2)) → true* *m.inside(Position(0, 3)) → false*
get(p: Position): T	El valor guardado en *p*. Si *p* está fuera, lanza *IndexOutOfBoundsException*.	*m.get(Position(1, 0)) → 4*
update(p: Position, value: T): Matrix[T]	Una matriz **nueva**, del mismo tamaño, en la que solo cambia el valor de *p*. La original no cambia. Si *p* está fuera, lanza *IndexOutOfBoundsException*.	*m.update(Position(1, 1), 99).data* *→ List(1, 2, 3, 4, 99, 6)*
partition(f: T => Boolean): (List[Position], List[Position])	Reparte las posiciones en dos listas, ambas en orden de filas: las que tienen un valor que cumple *f* y las que no. La matriz no cambia.	*m.partition(_ % 2 == 0)* *→ (List((0,1), (1,0), (1,2)),* *List((0,0), (0,2), (1,1)))*

Método	Qué debe cumplir	Ejemplo
neighbors(p: Position): List[Position]	Las posiciones vecinas de *p* (horizontal, vertical y diagonal) que están dentro de la matriz, sin repetir y sin incluir *p*. El orden no importa. Una esquina tiene 3 vecinas, un borde 5, una celda interior 8 y la única celda de una matriz 1x1 ninguna.	En una matriz 5x5: *neighbors(Position(0, 0))* → *(0,1) (1,0) (1,1)*
filterNeighbors(p: Position, f: T => Boolean): List[Position]	Las vecinas de *p* cuyo valor cumple *f*. La propia *p* no cuenta. El orden no importa.	*m.filterNeighbors(Position(0, 0), _ % 2 == 0)* → *(0,1) (1,0)* (valores 2 y 4)

Recursión de cola. El tablero del juego es de 15x15 (225 celdas), pero *Matrix* debe funcionar con cualquier tamaño y las pruebas usan una de 300x300 (90.000 celdas). Una recursión normal sobre una lista tan larga desborda la pila, así que los dos constructores que hay que escribir, *get*, *update* y *partition* deben ser recursivas de cola. Marcad vuestras funciones recursivas con *@annotation.tailrec*: el compilador os avisará si alguna no lo es.

Nota: *Matrix* no es una case class, por tanto, dos *Matrix* con los mismos datos no son *==*; para compararlas en vuestras pruebas, comparad sus *data*.

El tablero: GameOfLifeMatrix (1 pto)

El tablero del juego es una *Matrix[Cell]* con las operaciones propias del Juego de la Vida. Se define por **herencia**, como una case class que extiende de *Matrix[Cell]*. Sus parámetros sobrescriben (*override val*) los de la clase padre:

Elemento	Descripción
GameOfLifeMatrix(rows: Int, columns: Int, data: List[Cell])	La case class. Extiende de *Matrix[Cell]* y sus tres parámetros llevan *override val*, porque sobrescriben los *val* de la clase padre.
show(title: String = ""): String	Dibuja el tablero (ver más abajo). **Ya implementado.**
toString: String	Sobrescrito para que devuelva *show()*. **Ya implementado.**

Por ser case class, dos tableros con el mismo contenido son iguales y se pueden guardar en un *Set*, que es justo lo que necesitaremos para detectar cuándo se repite un estado. **Aprovechad lo que heredáis de *Matrix*.**

Método	Qué debe cumplir	Ejemplo
GameOfLifeMatrix.empty(rows: Int, columns: Int): GameOfLifeMatrix	Un tablero del tamaño indicado con todas las células muertas.	*GameOfLifeMatrix.empty(3, 4).aliveCount* → *0*
GameOfLifeMatrix.withAlive(rows: Int, columns: Int, alive: List[Position]): GameOfLifeMatrix	Un tablero con vivas exactamente las posiciones de la lista *alive* y el resto muertas.	*withAlive(5, 5, List((1,1), (1,2), (2,1), (2,2)))* es un bloque
update(p: Position, value: Cell): GameOfLifeMatrix	Lo mismo que el *update* heredado, pero devolviendo un *GameOfLifeMatrix* en lugar de una *Matrix[Cell]* (Scala permite acotar el tipo de retorno al sobrescribir).	*empty(3, 3).update(Position(1, 2), Alive)* es un *GameOfLifeMatrix* con una única célula viva
countAliveNeighbors(p: Position): Int	Cuántas vecinas de *p* están vivas. Vale igual para una celda viva que para una muerta.	Con el intermitente horizontal de la fila 2 de un 5x5: *(2,2)* → 2, *(1,2)* → 3, *(0,0)* → 0

Método	Qué debe cumplir	Ejemplo
aliveCells: List[Position]	Las posiciones de las células vivas, en orden de filas.	Un planeador → sus 5 posiciones
aliveCount: Int	Cuántas células vivas hay en el tablero.	Un planeador → *5*
nextGeneration: GameOfLifeMatrix	El tablero de la siguiente generación, aplicando las reglas B3/S23 a **todas** las celdas a la vez: sobreviven las vivas con 2 o 3 vecinas vivas y nacen las muertas con exactamente 3. El nuevo estado de cada celda se calcula siempre a partir del tablero actual, nunca de uno a medio actualizar. Las celdas de fuera del tablero cuentan como muertas. El tablero original no cambia.	El intermitente horizontal pasa a vertical y viceversa; el bloque no cambia; una L de 3 células se convierte en un bloque; una célula sola muere

El método *show* viene ya implementado en la plantilla y no hay que modificarlo. Devuelve el tablero como un *String* con los índices de filas y columnas, un marco y cada célula dibujada como un cuadrado: verde si está viva, gris oscuro si está muerta. El color se consigue con códigos ANSI, que funcionan en los terminales de IntelliJ, VS Code, macOS, Linux y Windows Terminal. Si recibe un título, lo escribe encima. Por ejemplo, *println(board.show("Jugador 1 · te quedan 3 células"))* con un planeador y un bloque en el tablero de 15x15 se ve así:



Opcional: las mismas matrices con los métodos de List (0,5 ptos)

Cuando tengáis *Matrix* y *GameOfLifeMatrix* funcionando, podéis escribir una segunda versión de ambas apoyándoos, ahora sí, en las funciones de orden superior de *List*: *MatrixList[T]*, una clase normal con los mismos métodos y contratos que *Matrix* (ver «Una clase Matrix genérica»), y *GameOfLifeMatrixList*, una case class que extiende de *MatrixList[Cell]* con los mismos métodos que *GameOfLifeMatrix* (ver «El tablero: GameOfLifeMatrix»). Sus constructores vienen ya implementados. El comportamiento debe ser idéntico al de la versión recursiva; *MatrixListTests.scala* lo comprueba comparando ambas versiones. Además, *MatrixList* tiene un método más, *cells: List[(Position, T)]*, que devuelve cada posición emparejada con su valor, en orden de filas. Os serán útiles, entre otras, *apply* (*data(i)*), *map*, *filter*, *partition*, *flatMap* y *zipWithIndex* de *List*.

En *GameOfLifeMatrixList* vienen ya implementados *toMatrix* (que la convierte en un *GameOfLifeMatrix*) y *toString*, que usa el *show* de *GameOfLifeMatrix*.

Nota: para la memoria, comparad las dos versiones. ¿Cuál es más corta? ¿Cuál os ha resultado más fácil de razonar? ¿Qué métodos de la versión recursiva se convierten en una sola línea? Fijaos también en que *GameOfLifeMatrix* y *GameOfLifeMatrixList* deberían quedar prácticamente iguales: lo único que cambia es la clase de la que heredan.

Jugadores y estado de la colocación (0,5 ptos)

Un enumerado indica de qué jugador se trata, y una case class guarda el estado de la fase de colocación: el tablero de cada jugador, las células que le quedan por colocar a cada uno y de quién es el turno. *State* son **solo datos**: toda la lógica que lo transforma está en la clase *Game*.

Elemento	Descripción
enum Player	Enumerado con dos casos: *Player.Player1* y *Player.Player2*.
otherPlayer(p: Player): Player	El rival de *p*: *otherPlayer(Player.Player1)* es *Player.Player2* y viceversa.
playerName(p: Player): String	El nombre que se muestra por pantalla: "Jugador 1" o "Jugador 2".
State(boards, remainingCells, turn)	Case class con tres campos: *boards: Map[Player, GameOfLifeMatrix]*, el tablero de cada jugador; *remainingCells: Map[Player, Int]*, las células que le quedan por colocar a cada uno; y *turn: Player*, de quién es el turno.

Un *State* nunca se modifica: cada cambio produce un estado nuevo (con *copy* y, para los *Map*, con *updated*).

Estrategias de colocación (1 pto)

En cada turno alguien tiene que decidir en qué casilla de **su propio tablero** se coloca la siguiente célula: el jugador humano tecleando una posición, o una IA. Para que la lógica del juego no tenga que distinguir entre ambos casos, las dos formas de jugar implementan una interfaz común:

Elemento	Descripción
trait Strategy	La interfaz común. Tiene un único método: *choosePosition(state: State, player: Player, error: Option[String]): Position*.
case object ManualStrategy	La estrategia del jugador humano: pide la posición por teclado.
case class AIStrategy(chosenPattern: List[Position])	La estrategia automática (parte opcional).

El contrato de *choosePosition*. Recibe el estado actual, el jugador que debe colocar y un *Option[String]* con el motivo por el que se rechazó su posición anterior: *None* si no hubo error (el caso normal) y *Some(mensaje)* si la posición anterior no era válida y se le vuelve a preguntar. Devuelve una posición del tablero de ese jugador. La estrategia **no comprueba** si la posición es válida: de eso se encarga *Game.placeCell* (ver «La clase Game: fase de colocación»).

ManualStrategy es la estrategia del jugador humano y la única parte de la colocación con entrada/salida. Cada vez que se le pide una posición debe:

- Si *error* es *Some(mensaje)*, mostrar el mensaje para que el jugador sepa por qué se ha rechazado su posición.

- Mostrar el tablero del jugador con *show*, con un título que diga de quién es el turno y cuántas células le quedan.
- Leer una línea con *StdIn.readLine* en el formato "fila columna" y convertirla en una *Position*. Si el texto no tiene ese formato (no son dos números enteros), debe avisar y volver a leer ella misma, de forma recursiva, sin devolver nada hasta tener una posición bien escrita. Si *readLine* devuelve *null* (se ha cerrado la entrada), la partida termina.

Por ejemplo, un jugador que se equivoca dos veces antes de acertar:

El jugador escribe	Qué ocurre
dos tres	*ManualStrategy* ve que no son dos números enteros, avisa de que el formato no es válido y vuelve a leer, sin devolver nada todavía.
7 1	*ManualStrategy* devuelve *Position(7, 1)*. *placeCell* la rechaza con *Left("La posición (7, 1) está fuera del tablero.")*, y *placementLoop* vuelve a llamar a *choosePosition* con ese mensaje dentro de *Some*. *ManualStrategy* muestra el mensaje y dibuja de nuevo el tablero.
2 1	La posición es válida: *placeCell* devuelve *Right*, la célula queda colocada y el turno pasa al rival.

Nota: Fijaos en que los dos errores los detectan piezas distintas: el de formato, la propia estrategia; el de validez, *placeCell*, que es quien conoce las reglas.

La clase Game: fase de colocación (2 ptos)

La clase *Game* guarda la configuración de una partida y contiene **toda su lógica**. Sus métodos son funciones puras: no leen de teclado ni escriben por pantalla, porque eso es tarea de las estrategias y del programa principal. En esta sección se describen los métodos de la fase de colocación; los de la simulación están en la siguiente.

Método	Resumen
Game(rows: Int, columns: Int, budget: Int, maxGenerations: Int)	La clase y su configuración. En el programa principal siempre será *Game(15, 15, presupuesto, 1000)*; las pruebas usan también tableros más pequeños.
initialState: State	El estado al empezar la colocación.
hasFinished(state: State, player: Player): Boolean	Si el jugador ya ha colocado todas sus células.
placeCell(state: State, p: Position): Either[String, State]	Intenta colocar una célula del jugador de turno.
placementLoop(state: State, strategies: Map[Player, Strategy]): State	El bucle de la fase de colocación.
simulate(board: GameOfLifeMatrix): (Int, Int)	Simula un tablero hasta que se estabiliza.
determineWinner(resultPlayer1: (Int, Int), resultPlayer2: (Int, Int)): Option[Player]	Decide quién gana.
play(strategies: Map[Player, Strategy]): GameResult	Juega una partida completa.

initialState devuelve el estado de partida: dos tableros vacíos de *rows* x *columns*, *budget* células por colocar para cada jugador y el turno para *Player1*.

hasFinished(state, player) es verdadero si a ese jugador no le quedan células por colocar.

placeCell(state, p) intenta colocar una célula viva del jugador de turno en la posición *p* de **su** tablero. El resultado es un *Either*, tal y como vimos en el tema de gestión de errores: *Left(mensaje)* si la jugada no es válida y *Right(nuevoEstado)* si lo es. La jugada no es válida en tres casos:

- El jugador de turno ya no tiene células por colocar.
- La posición está fuera del tablero.
- La casilla ya tiene una célula viva.

Si la jugada es válida, el nuevo estado tiene la célula viva en el tablero del jugador de turno (el del rival no cambia) y una célula menos en su presupuesto. El turno pasa al rival, **salvo que el rival ya haya terminado**: en ese caso sigue siendo del mismo jugador. En los ejemplos, *game = Game(4, 5, 3, 100)* y *s0 = game.initialState*:

Llamada	Resultado
game.placeCell(s0, Position(1, 1))	*Right(s1)*: *Player1* tiene viva la *(1,1)*, le quedan 2 células, a *Player2* le quedan 3 y el turno es de *Player2*.
game.placeCell(s0, Position(4, 0))	*Left(...)*: la fila 4 no existe en un tablero de 4 filas.
game.placeCell(s1, Position(1, 1))	*Right(...)*: ahora es el turno de *Player2*, y en **su** tablero la *(1,1)* está libre.
placeCell sobre un estado con *turn = Player1*, a *Player1* le quedan 2 y a *Player2* 0	*Right(...)* con el turno otra vez para *Player1*, porque su rival ya ha terminado.

placementLoop(state, strategies) es el bucle de la fase de colocación, y debe ser recursivo de cola. Recibe el estado inicial y la estrategia de cada jugador, y devuelve el estado cuando los dos jugadores han colocado todas sus células. Debe cumplir lo siguiente:

- A un jugador que ya no tiene células por colocar **nunca se le piden coordenadas**, aunque sea su turno: el turno pasa al rival.
- Las posiciones se piden a la estrategia del jugador de turno y se validan con *placeCell*. Si una posición no es válida, se vuelve a pedir **al mismo jugador**, pasándole como error *Some(mensaje)* con el motivo (ver «Estrategias de colocación»). En cualquier otro caso, el error que recibe la estrategia es *None*.

Con un presupuesto de 0, *placementLoop* devuelve el estado sin preguntar a nadie.

Nota: fijaos en que *placementLoop* no sabe si la estrategia de cada jugador es manual o una IA: simplemente llama a *choosePosition*. Eso es lo que nos da el polimorfismo. Y como *placeCell* devuelve un *Either* en lugar de lanzar una excepción o imprimir un mensaje, la lógica del juego se puede probar sin teclado ni pantalla.

La clase Game: simulación, ganador y partida completa (1 pto)

simulate(board) hace evolucionar un tablero, generación a generación, hasta que se estabiliza, y devuelve una tupla *(generaciones, células vivas)*. Se detiene en cuanto el tablero vuelve a un estado que ya ha tenido en esta simulación (esto cubre tanto un estado fijo como un ciclo de cualquier longitud), o cuando llega a *maxGenerations*. El tablero inicial es la generación 0, y el resultado es el número de la generación en la que se detiene junto con las células vivas que tiene en ese momento. Debe ser recursiva de cola.

Por ejemplo, el bloque da *(1, 4)*: el tablero de la generación 1 es igual que el de la generación 0, que ya se había visto. Otros ejemplos, con *maxGenerations = 100* salvo que se diga otra cosa:

Tablero inicial	Resultado	Por qué
Bloque	*(1, 4)*	No cambia nunca: se repite en la generación 1.
Intermitente	*(2, 3)*	Tiene período 2: en la generación 2 vuelve a estar como al principio.
Tablero vacío	*(1, 0)*	Como el bloque: el vacío se repite en la generación 1.
Una célula sola	*(2, 0)*	Muere en la generación 1 y el vacío se repite en la 2.
Planeador, *maxGenerations = 3*	*(3, 5)*	Se para por el límite, con el planeador aún vivo.
Planeador en un 8x8, *maxGenerations = 500*	menos de 500 generaciones, 4 o 0 vivas	Acaba chocando contra el borde.

determineWinner(r1, r2) recibe los resultados de los dos jugadores y devuelve quién gana: primero cuentan las generaciones y, a igualdad de generaciones, las células vivas. Si también hay empate en las células vivas, nadie gana y el resultado es *None*. Por ejemplo: *determineWinner((10, 3), (7, 50))* → *Some(Player.Player1)*; *determineWinner((5, 4), (5, 8))* → *Some(Player.Player2)*; *determineWinner((5, 4), (5, 4))* → *None*.

play(strategies) juega una partida completa: la fase de colocación desde *initialState*, la simulación del tablero de cada jugador y el cálculo del ganador. Devuelve un *GameResult*:

Campo	Contenido
finalState: State	El estado tras la fase de colocación, con los patrones iniciales de ambos jugadores.
results: Map[Player, (Int, Int)]	El resultado de *simulate* para cada jugador: (generaciones, células vivas).
winner: Option[Player]	El ganador, o *None* si hay empate total.

Ejemplo que podéis probar: en un tablero de 5x5 con 4 células por jugador: *Player1* coloca un bloque y obtiene *(1, 4)*; *Player2* coloca una célula en cada esquina, que mueren en la primera generación, y obtiene *(2, 0)*. **Gana *Player2***, porque su tablero tarda una generación más en estabilizarse, aunque termine sin ninguna célula viva.

La Vida, asistida y dirigida por una IA

Estrategia dirigida por IA (1 pto, opcional)

Esta parte consiste en diseñar una estrategia automática, razonarla en la memoria y programarla como una segunda implementación de *Strategy*, hermana de *ManualStrategy*. Como para evaluar un patrón hay que

conocerlo entero (no se puede simular un tablero a medio colocar), la IA lo calcula **una sola vez, antes de empezar**, y luego lo va sirviendo posición a posición, turno a turno:

Elemento	Descripción
AIStrategy(chosenPattern: List[Position])	La case class: guarda el patrón elegido y lo sirve turno a turno.
AIStrategy(game: Game, attempts: Int): AIStrategy	El método *apply* del objeto compañero: busca el patrón y construye la estrategia.

- *chosenPattern* tiene exactamente *budget* posiciones, distintas y dentro del tablero.
- *choosePosition* devuelve la siguiente posición del patrón que falta por colocar. Es decir, en cada turno del jugador sirve la siguiente posición del patrón, por orden. Ignora el parámetro *error*, porque sus posiciones nunca deberían ser inválidas; si lo fueran, *placementLoop* le preguntaría una y otra vez.
- *AIStrategy(game, attempts)* busca el patrón y construye la estrategia. *attempts* debe ser al menos 1.

Una heurística sencilla y efectiva es la **búsqueda aleatoria**: generar *attempts* patrones al azar, simular cada uno con *game.simulate* y quedarse con el que obtenga mejor resultado. Es decir, reutilizamos la propia simulación como función de evaluación, algo muy habitual en las heurísticas de búsqueda. Para generar patrones al azar podéis usar *Random*.

Nota: no es obligatorio usar búsqueda aleatoria: podéis proponer cualquier otra heurística razonada (por ejemplo, favorecer patrones conocidos por ser longevos, o formas compactas frente a dispersas). Lo importante es que expliquéis en la memoria qué pretende medir, cómo la aproxima y en qué casos puede fallar.

Programa principal (0,5 ptos)

El programa principal es el objeto *Main*, que extiende de *App*: el código que escribáis en su cuerpo es lo que se ejecuta al arrancar el programa. En la plantilla vienen ya las dos constantes de la partida (*BoardSize = 15* y *MaxGenerations = 1000*) y las firmas de algunas funciones auxiliares sugeridas. Tenéis libertad para diseñarlo, pero debe cumplir lo siguiente:

- Usar siempre un tablero de **15x15** y un máximo de **1000 generaciones** (definidlos como constantes en *Main.scala*), y pedir el presupuesto de células por jugador, entre 1 y 225, volviendo a preguntar si el valor no es válido.
- Permitir que cada jugador sea humano (*ManualStrategy*) o, si habéis hecho la parte opcional, una IA (*AIStrategy*), simplemente construyendo el *Map[Player, Strategy]* adecuado.
- Jugar la partida con *game.play* y mostrar los patrones iniciales de ambos jugadores con *show* (ver «El tablero: GameOfLifeMatrix»), sus resultados (generaciones y células vivas) y quién ha ganado y por qué (por generaciones o por células vivas), o que ha habido empate.
- Ofrecer ver la simulación animada, generación a generación, con *Animation.animate(result)*.

El fichero *Animation.scala* viene **ya implementado** en la plantilla y no hay que modificarlo. *Animation.animate(result)* limpia la consola entre una generación y la siguiente y dibuja los dos tableros evolucionando a la vez, con un título que indica la generación y las células vivas de cada uno. Cada tablero avanza durante las generaciones que calculó vuestro *simulate* y después se queda quieto marcado como estabilizado; los fotogramas se obtienen con vuestro *nextGeneration*. Es decir, la animación muestra exactamente lo que ha calculado vuestro código: si *nextGeneration* o *simulate* tienen un error, se verá. Además,

Animation.sideBySide(izquierdo, derecho) coloca dos tableros ya dibujados uno al lado del otro y *Animation.clearScreen()* borra la terminal.

Nota: la animación borra la terminal y redibuja cada generación encima de la anterior mediante códigos ANSI, de modo que en pantalla solo hay un tablero por jugador. Funciona en cualquier terminal (macOS, Linux, Windows Terminal, o el terminal integrado de IntelliJ y de VS Code). La ventana de ejecución (Run) de IntelliJ, en su modo normal, solo entiende los colores y mostraría las generaciones una debajo de otra. Para que también allí se vea la animación, activad la emulación de terminal: Run > Edit Configurations..., elegid la configuración del programa, Modify options > Emulate terminal in output console.

Opcional: gestión de efectos (0,5 ptos)

Podéis completar la práctica encapsulando las interacciones de entrada/salida y los cálculos que puedan fallar (por ejemplo, leer una posición inválida por teclado) usando los tipos vistos en el tema de gestión de efectos. Explicad en la memoria las decisiones tomadas.

Nota: como extra opcional y no puntuable, si os sobran ganas podéis abstraer también la propia regla del autómata detrás de un trait (por ejemplo, *trait AutomatonRule* con los métodos *survives(aliveNeighbors: Int)* e *isBorn(aliveNeighbors: Int)*, y *object ConwayRule extends AutomatonRule* implementándolo para B3/S23). Así *nextGeneration* podría recibir la regla como parámetro y soportar variantes reales del autómata, como HighLife (B36/S23), que tiene la curiosidad de incluir un patrón capaz de autorreplicarse. Contadlo en la memoria si os animáis.

Programando progresivamente la práctica

Os recomendamos avanzar en este orden, probando cada pieza antes de pasar a la siguiente: *Matrix*, *GameOfLifeMatrix* (primero con patrones conocidos como el intermitente o el bloque), *Player* y *State*, la fase de colocación de *Game* con estrategias de prueba, la simulación y, al final, *ManualStrategy* y el programa principal. La forma más rápida de probar una función suelta es un objeto propio que extienda de *App* y en cuyo cuerpo la vayáis invocando con matrices pequeñas (por ejemplo, de 5x5).

Para comprobar vuestras implementaciones os proporcionamos unos ficheros de pruebas con *assert*, sin ningún framework. Cada fichero es un objeto que extiende de *App* y se ejecuta como cualquier programa. Al ejecutarlo, cada prueba muestra OK si pasa o XX si falla (con el motivo), y las funciones que todavía tienen *???* aparecen como pendientes. La plantilla compila tal cual, así que podéis ejecutar las pruebas desde el primer momento e ir viendo cómo pasan a medida que completáis funciones. Las pruebas no se entregan; para que compilen, vuestro código debe respetar los nombres y las firmas de la plantilla.

Fichero	Se ejecuta con	Comprueba
TestKit.scala	—	Utilidades de las pruebas. No hay que tocarlo.
MatrixTests.scala	*MatrixTests*	*Matrix*, incluida la recursión de cola.
GameOfLifeMatrixTests.scala	*GameOfLifeMatrixTests*	*GameOfLifeMatrix*: *update*, vecinas vivas, igualdad y reglas del juego.
MatrixListTests.scala	*MatrixListTests*	Opcional: *MatrixList* y *GameOfLifeMatrixList*.

Podéis añadir todos los test que queráis.

Entrega de la práctica

Hay que subir dos archivos: un fichero **.zip** con todos los ficheros. scala de la práctica (con la misma estructura que la plantilla y sin las pruebas) y un fichero **PDF** con una breve memoria. El código debe incluir comentarios de las funciones y de las partes más complejas, y un programa principal que ponga en juego todo lo creado. La memoria debe ser muy breve y debe incluir:

- **Decisiones y dificultades encontradas:** las decisiones que hayáis tomado para estructurar el código o programar alguna función relevante (por ejemplo, cómo habéis conseguido que *partition* sea recursiva de cola, o cómo habéis organizado *nextGeneration*), y cómo habéis resuelto cualquier problema. No hace falta incluir capturas de código ni listar funciones.
- **Heurística elegida:** si habéis hecho la parte de IA, explicad qué mide, cómo la aproxima y en qué casos puede no funcionar bien.
- **Extras:** si habéis programado algo adicional que consideréis relevante. Por ejemplo, la versión de las matrices con los métodos de *List*.
- **Coordinación y reparto:** cómo ha sido la coordinación y el reparto de trabajo entre los integrantes del grupo.

Una práctica entregada fuera de plazo se calificará como suspensa. Si se entrega con un retraso de menos de 24 horas, se penalizará con un 30 % de la nota final.

La puntuación es la indicada en cada apartado. Se podrá penalizar la nota total de la práctica si el código no sigue una estructura correcta o si no está documentado, o lo está en exceso. Si se sospecha que la práctica se ha hecho con inteligencia artificial, se seguirá el protocolo establecido y se realizará una evaluación individual en la que cada estudiante deberá defender su implementación.